



最优化理论

第13讲 现代优化方法(1) 蚁群算法

辽宁大学 信息学院



2023年9月25日星期一

遼寧大學

现代优化方法

1. 组合优化问题
2. 启发式算法
 - ① 蚁群算法
 - ② 遗传算法
 - ③ 模拟退火算法

组合优化问题

 组合优化问题是利用数学方法去寻找离散事件的最优化编排、调度、分组、排序以及筛选等相关问题。是目前计算机理论中非常重要的研究内容。

 常见的组合优化问题：

 旅行商问题（Travelling salesman problem, TSP）

 0-1背包问题（Knapsack problem）

 图着色问题（Graph Coloring Problem, GCP）

组合优化问题

 组合优化问题中主要研究对象为NP完全问题的求解。

 P问题

 NP问题

 NPC问题

 NP-Hard问题

组合优化问题

计算复杂性理论

 计算复杂性理论（Computational complexity theory）研究计算问题时所需的资源，比如时间和空间，以及如何尽可能的节省这些资源。

 时间复杂度和空间复杂度决定了算法的优劣

 组合优化问题重点分析时间复杂度

 其中复杂度的含义是随着问题规模的扩大，其时间的增长趋势，并非绝对的程序执行时间

组合优化问题

计算复杂性理论

多项式时间

- 🖱️ 多项式时间（Polynomial time）一般作为算法高效的标志
- 🖱️ 一个算法的时间复杂度是多项式时间可以表示为 $O(p)$ ，其中

$$p = n^K + n^{K-1} + \dots$$

k为任意整数

组合优化问题

计算复杂性理论

 **P问题 (Polynomial Problems)** 指的是存在一个多项式时间算法的求解问题

 **NP问题 (Non-Deterministic Polynomial Problems)** 指的是可以在多项式时间内**验证**它的解是否正确的问题

 **P 问题一定是一个NP问题，反之不一定**

 **$P=NP$? 是一个未解之谜**

组合优化问题

计算复杂性理论

-  NP-Hard问题指的是所有的NP问题都可以在多项式时间内规约（Reduce）到该问题。
-  如果一个NP-Hard问题还是一个NP问题，则这个问题就是NP-Complete（NPC），也就是NP完全问题。
-  NPC问题一定是NP-Hard问题，反过来不成立。

组合优化问题

TSP问题

 旅行商问题，即TSP问题（Traveling Salesman Problem）。假设有一个旅行商人要拜访 n 个城市，他必须选择所要走的路径，路径的限制是每个城市只能拜访一次，而且最后要回到原来出发的城市。路径的选择目标是要求得的路径路程为所有路径之中的最小值。

组合优化问题

TSP问题

 TSP问题已经被证明是一个NPC问题。

 TSP问题的优化模型描述为：

 目标：走过的路径最小化

 约束：

- 1. 一个城市只能走一次
- 2. 出发点和终点是相同的
- 3. 所有城市都需要走一遍

组合优化问题

TSP问题

 目标：走过的路径最小化

假设城市个数为 N ，城市 i 和城市 j 之间的距离为 d_{ij}

$$\min \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} d_{ij} x_{ij}$$

$$x_{ij} = \begin{cases} 1 & i \rightarrow j \text{ 被选中} \\ 0 & i \rightarrow j \text{ 未被选中} \end{cases}$$

组合优化问题

TSP问题

 约束1：一个城市只能走一次

$$\forall i \in \{0, 1, 2, \dots, N-1\}$$

$$\sum_{j=0}^{N-1} x_{ij} = 1$$

这个条件也表示了约束3：所有城市都走一遍。

$$\forall j \in \{0, 1, 2, \dots, N-1\}$$

$$\sum_{i=0}^{N-1} x_{ij} = 1$$

组合优化问题

TSP问题

 约束2：出发点和终点是相同的

$$\forall 0 < i \neq j \leq N-1, u_i \in R, u_j \in R$$

$$u_i - u_j + nx_{ij} \leq n-1$$

该约束来自于**TSP**的**MTZ**（**Miller-Tucker-Zemlin formulation**）模型。

通过对除首节点外消除子环来保证出发点和终点是相同的。

组合优化问题



TSP问题



数学定义

$$\min \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} d_{ij} x_{ij}$$

$$s.t. \left\{ \begin{array}{l} \forall i \in \{0, 1, 2, \dots, N-1\} \\ \sum_{j=0}^{N-1} x_{ij} = 1 \\ \forall j \in \{0, 1, 2, \dots, N-1\} \\ \sum_{i=0}^{N-1} x_{ij} = 1 \\ \forall 0 < i \neq j \leq N-1, u_i \in R, u_j \in R \\ u_i - u_j + nx_{ij} \leq n-1 \\ x_{ij} = \begin{cases} 1 & i \rightarrow j \text{ 被选中} \\ 0 & i \rightarrow j \text{ 未被选中} \end{cases} \end{array} \right.$$

组合优化问题

组合优化问题求解思路

 首先证明它是一个NPC问题或者NP-Hard问题

 根据最优化四个基本算法思想进行算法设计

 1.动态规划

 2.分治

 3.贪心

 4.回溯

 如果以上优化算法设计难度很大，则可以采用近似解算法来实现，启发式算法就是常用的方法。

组合优化问题

启发式算法

 通过启发式信息来进行求解，算法设计简单，通常得到的是近似解，但是可以解决工程问题。

 分类：

 蚁群算法

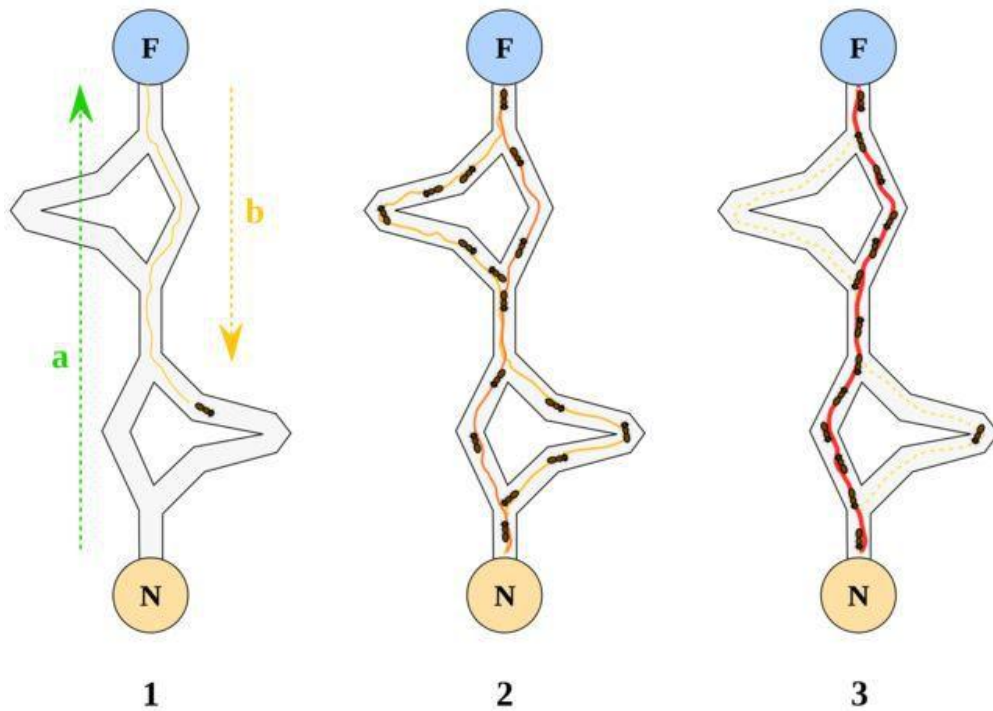
 遗传算法

 模拟退火算法

 神经网络算法

启发式算法—蚁群算法

🖥️ 蚂蚁几乎没有视力，但他们却能够在黑暗的环境下找到食物，而且能够找到一条从洞穴到食物的**最短路径**。它们是如何做到的呢？



启发式算法—蚁群算法

-  蚁群算法(Ant Colony Optimization, ACO), 又称蚂蚁算法, 由马克·多里戈(Marco Dorigo)于1992年在他的博士论文中提出。是一种用来在图中**寻找优化路径**的机率型算法。
-  其灵感来源于蚂蚁在寻找食物过程中发现路径的行为。
-  最初用于解决**旅行商问题(Traveling Salesman Problem, TSP)**。

启发式算法—蚁群算法

 蚁群优化算法最初用于解决**TSP问题**，经过多年的发展，已经陆续渗透到其他领域中：

 比如图着色问题、大规模集成电路设计、通讯网络中的路由问题以及负载平衡问题、车辆调度问题等。

 蚁群算法在若干领域已获得成功的应用，其中最成功的是在组合优化问题中的应用

启发式算法—蚁群算法

基本原理

-  蚂蚁在行走过程中会释放一种称为“**信息素**”的物质，用来标识自己的行走路径。
-  在寻找食物的过程中，**根据信息素的浓度选择行走的方向**，并最终到达食物所在的地方。
-  **信息素会随着时间的推移而逐渐挥发。**

蚁群算法原理示意图

如图，假设有A, B, C, D, E, H五个节点，其中A为蚁穴，E为食物源。B, C, D, H四个点为路径上的途经点。

一只蚂蚁从A到E，只有两条路径可供选择：

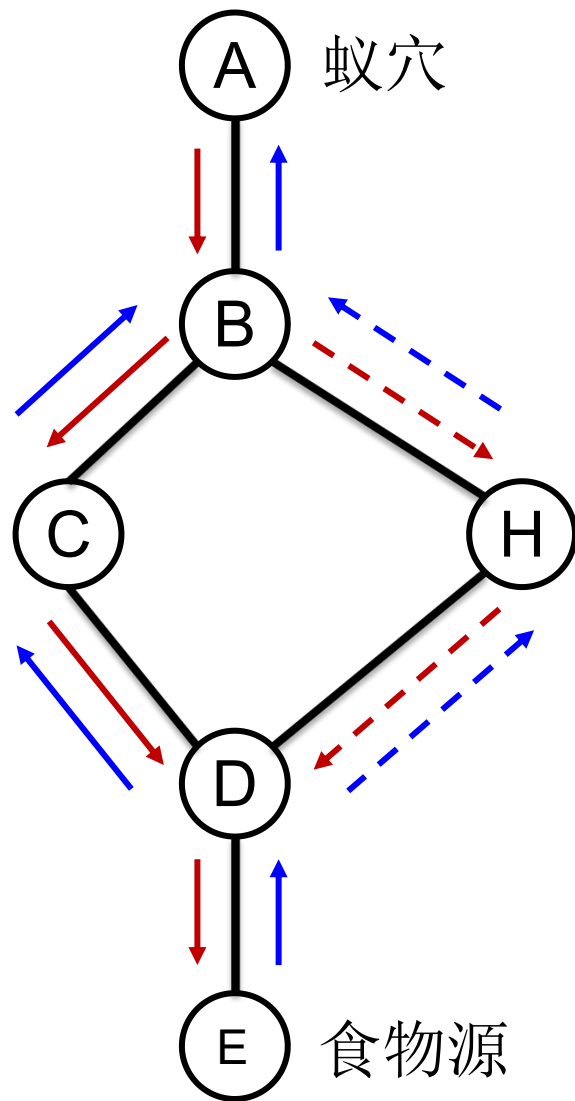
路径1：A-B-C-D-E

路径2：A-B-H-D-E

从E点返回到A也同样只有两条路径可供选择：

路径1：E-D-C-B-A

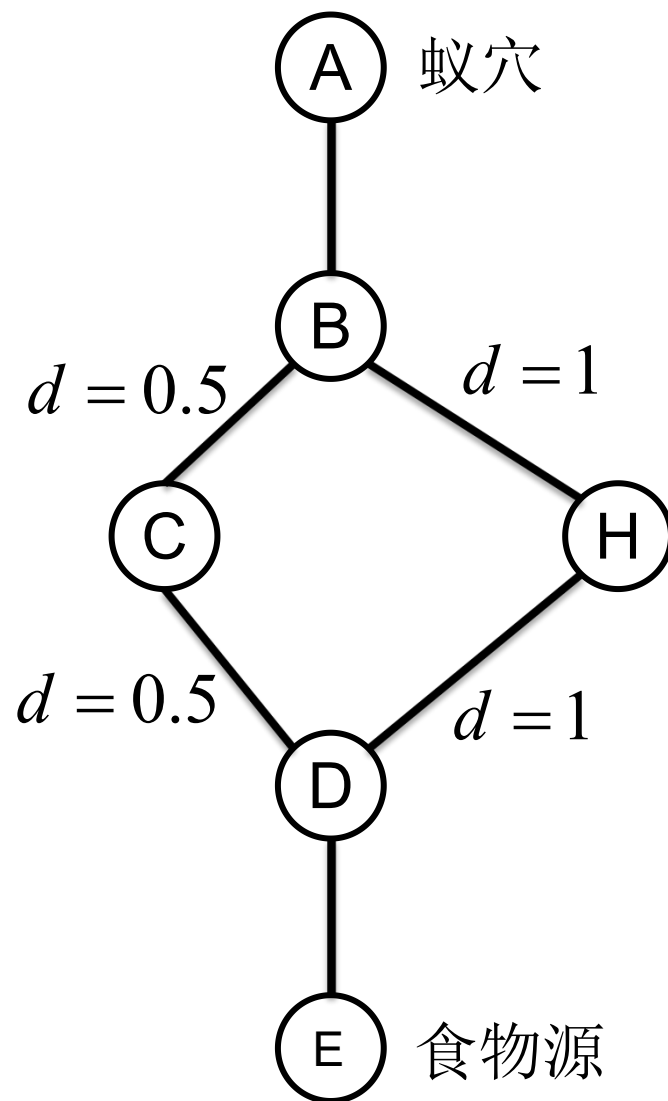
路径2：E-D-H-B-A



蚁群算法原理示意图

假设不同路径的距离不同，图中B-C，C-D的路径长度为0.5个单位长度，而B-H，H-D的路径长度为1个单位长度。

假设每经过1个单位时刻，就有30只蚂蚁从A和E同时出发，它们的速度是每单位时刻行进1个单位长度。



蚁群算法原理示意图

时刻1

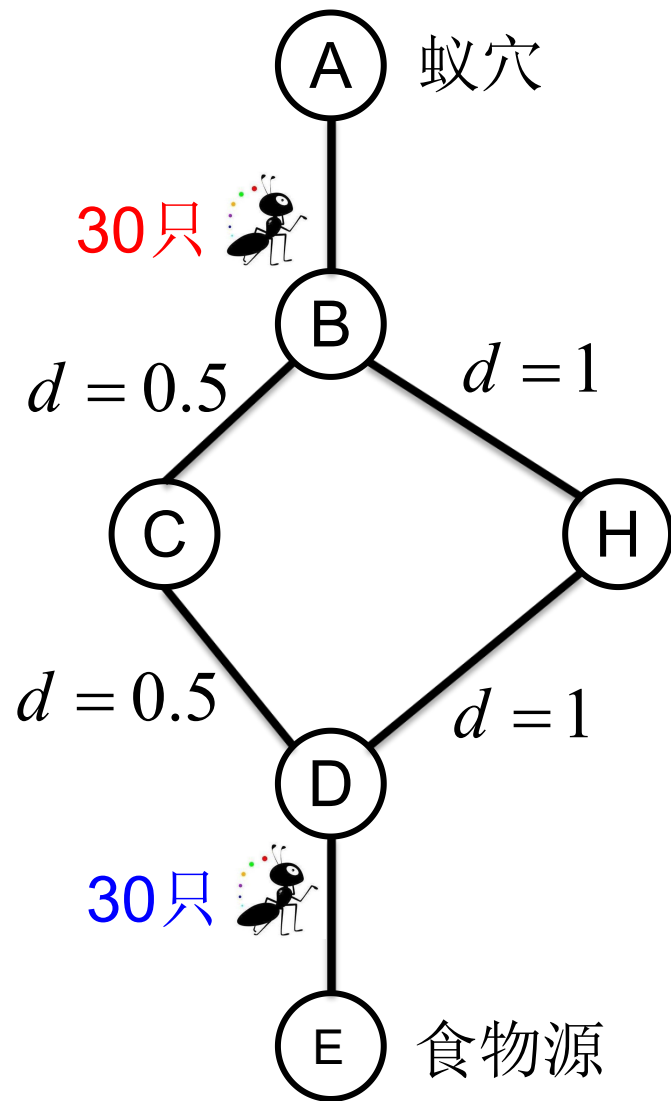
30只蚂蚁从A出发行进到B

30只蚂蚁从E出发行进到D

在B点，由于当前两条备选路径都没有任何参照物（信息素），因此蚂蚁选路是完全随机的。

15只走BCD, 15只走BHD

同理，在D点，15只走DCB, 15只走DHB



蚁群算法原理示意图

时刻2

从B点出发走BCD的15只蚂蚁到达了D, 另外15只走BHD的到达了H。

同理, D点出发走DCB的15只蚂蚁到了B, 另外15只走DHB的到达了H。

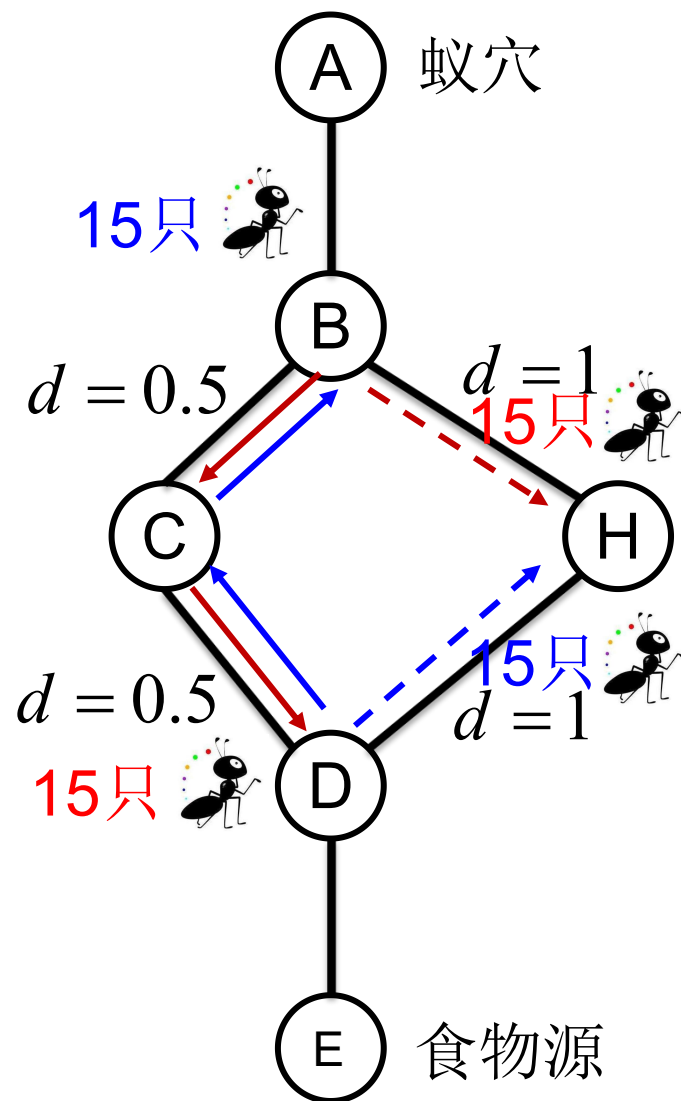
因此, 这些蚂蚁分别给路径B-C, C-D, B-H、HD释放了一份信息素, 具体如下:

B-C信息素: 15份+15分=30份

C-D信息素: 15份+15分=30份

B-H信息素: 15份

H-D信息素: 15份



蚁群算法原理示意图

时刻2

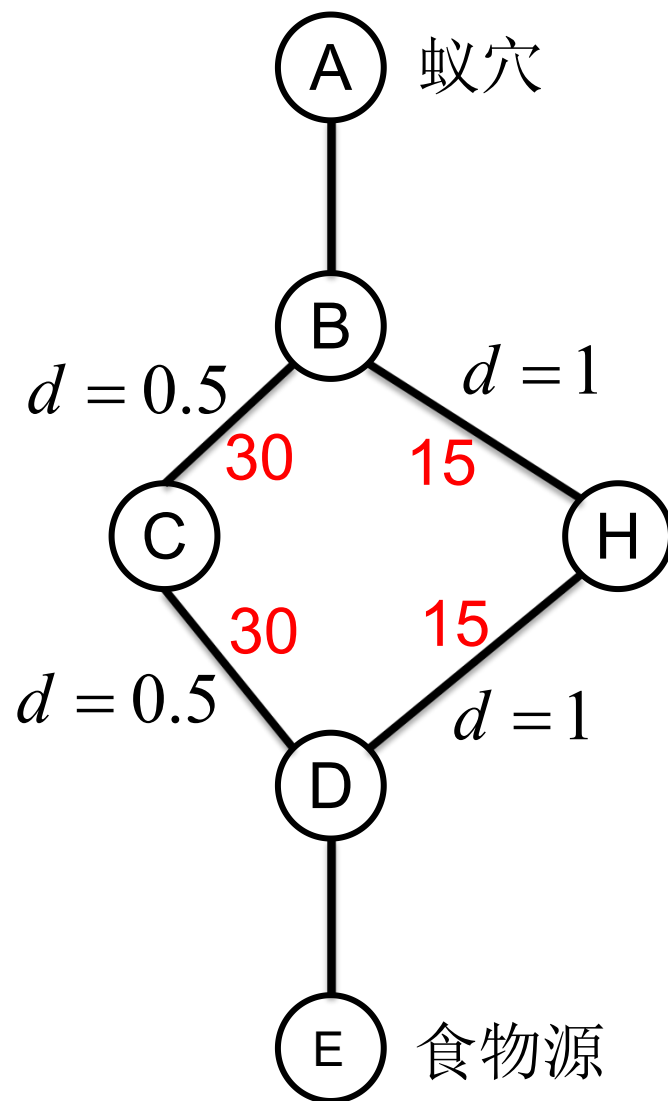
B-C信息素: 15份+15分=30份

C-D信息素: 15份+15分=30份

B-H信息素: 15份

H-D信息素: 15份

此时, 又有30只蚂蚁到达B和D, 则它们会根据现有的浓度进行选择, 经过多次迭代以后, **BCD**上的信息素浓度越来越大, **BHD**上的信息素随着挥发越来越小, 直到为0。最后所有的蚂蚁都会选择**BCD**进行食物搬运



启发式算法—蚁群算法

技术难点

如何选择路径

🖱️ 选择的依据是信息素浓度还是局部化的路径长度？

如何更新信息素

🖱️ 如何确新增加信息素和挥发信息素之间的关系？

启发式算法—蚁群算法

如何选择路径

 假设在 t 时刻，第 k 只蚂蚁从节点 i 出发，则它选择下一个节点 j 的概率用如下公式表示：

$$p_{ij}^k(t) = \frac{\left(\tau_{ij}(t)\right)^{\alpha} \left(\eta_{ij}(t)\right)^{\beta}}{\sum_{j \in N_i^k} \left(\tau_{ij}(t)\right)^{\alpha} \left(\eta_{ij}(t)\right)^{\beta}}$$

蚁群算法选路概率公式

$$p_{ij}^k(t) = \frac{\left(\tau_{ij}(t)\right)^\alpha \left(\eta_{ij}(t)\right)^\beta}{\sum_{j \in N_i^k} \left(\tau_{ij}(t)\right)^\alpha \left(\eta_{ij}(t)\right)^\beta}$$

$\tau_{ij}(t)$ t 时刻节点*i*和*j*之间路径上的信息素的浓度，由算法提供。

$\eta_{ij}(t)$ t 时刻节点*i*和*j*之间的启发信息，不同问题可以有不同含义
(在解决TSP问题中，表示节点*i*和*j*之间路径的长度的倒数)

α 信息素的重要程度

β 启发信息的重要程度

N_i^k 所有 t 时刻节点*i*出发，可以访问的下一个节点集合

启发式算法—蚁群算法

如何选择路径

 系统产生一个随机数，这个随机数根据选路的概率来最终确定走哪一条路径。

 注：不是一定选择概率最高的那一条

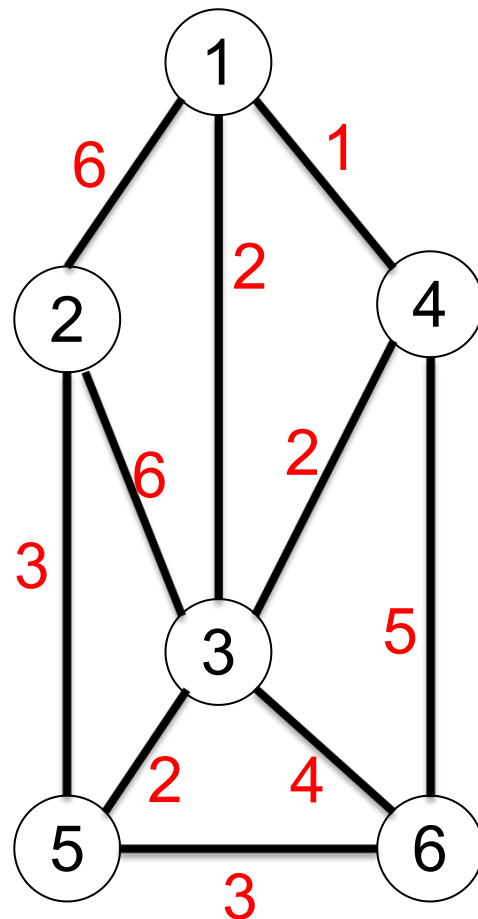
蚁群算法路径选择示例

如图可知每条路径上的数字代表每条路径的距离 d_{ij} 。
假设初始时每条路径上的信息素浓度均为1，信息素因子 α 为2，启发函数因子 β 为3
由一只蚂蚁 k 的起点为城市2到下一个城市 j 的概率分别为：

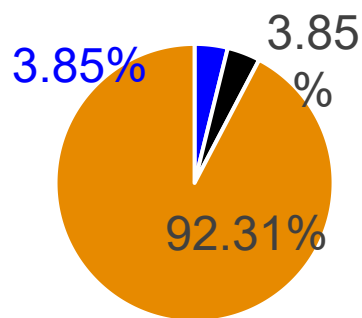
$$P_{21}^k = \frac{\tau_{2j}^2(t) * \eta_{2j}^3(t)}{\sum_{s \in N_k} \tau_{2j}^2(t) * \eta_{2j}^3(t)} = \frac{1^2/6^3}{\frac{1^2}{6^3} + \frac{1^2}{6^3} + \frac{1^2}{3^3}} = \frac{1}{26}$$

$$P_{23}^k = \frac{\tau_{2j}^2(t) * \eta_{2j}^3(t)}{\sum_{s \in N_k} \tau_{2j}^2(t) * \eta_{2j}^3(t)} = \frac{1^2/6^3}{\frac{1^2}{6^3} + \frac{1^2}{6^3} + \frac{1^2}{3^3}} = \frac{1}{26}$$

$$P_{25}^k = \frac{\tau_{2j}^2(t) * \eta_{2j}^3(t)}{\sum_{s \in N_k} \tau_{2j}^2(t) * \eta_{2j}^3(t)} = \frac{1^2/3^3}{\frac{1^2}{6^3} + \frac{1^2}{6^3} + \frac{1^2}{3^3}} = \frac{24}{26}$$



蚁群算法路径选择示例



■ 到节点1 ■ 到节点3 ■ 到节点5

轮盘赌法:

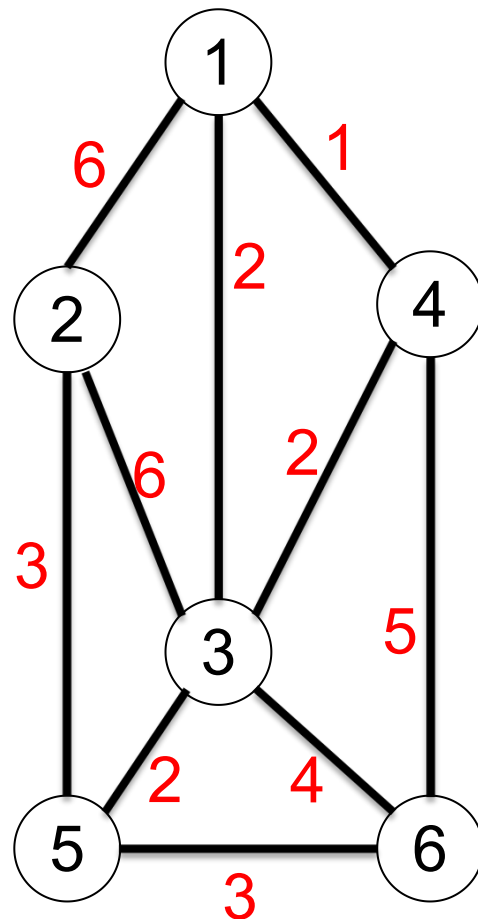
1. $[0-0.0384]$
2. $[0.0385-0.076]$
3. $[0.077-1)$

产生一个随机数:

若为0.03, 则选择节点1。

若为0.05, 则选择节点3。

若为0.20, 则选择节点5。



启发式算法—蚁群算法

如何更新信息素

 为了避免过多信息素残留引起的浓度淹没启发式信息的问题，要进行挥发，公式如下：

$$\tau_{ij}(t+n) = (1-\rho)\tau_{ij}(t) + \sum_{k=1}^m \Delta\tau_{ij}^k$$

蚁群算法信息素更新公式

$$\tau_{ij}(t+n) = (1-\rho)\tau_{ij}(t) + \sum_{k=1}^m \Delta\tau_{ij}^k$$

$\tau_{ij}(t+n)$ $t+n$ 时刻节点 i 和 j 之间路径上的信息素的浓度。

$\tau_{ij}(t)$ t 时刻节点 i 和 j 之间路径上信息素的浓度

ρ 信息素挥发因子 ($1-\rho$ 就是保留因子)

m 蚂蚁的数量

$\sum_{k=1}^m \Delta\tau_{ij}^k$ 所有 t 到 $t+n$ 时刻节点 i 和 j 之间路径上增加的信息素浓度和

启发式算法—蚁群算法

如何更新信息素

 根据不同的规则我们可以将蚁群算法分为三种模型：

-  蚁周模型(Ant-Cycle)

-  蚁密模型(Ant-Density)

-  蚁量模型(Ant-Quantity)

启发式算法—蚁群算法

如何更新信息素

 根据不同的规则我们可以将蚁群算法分为三种模型：

蚁周模型(Ant-Cycle)

- 如果在一次循环中，蚂蚁k经过路径ij，则

$$\Delta \tau_{ij}^k = \frac{Q}{L_k}$$

Q 是一个常量
 L_k 是所有经过路径总和

- 更新原则：所有m只蚂蚁都走完所有节点后进行更新

启发式算法—蚁群算法

如何更新信息素

 根据不同的规则我们可以将蚁群算法分为三种模型：

蚁密模型(Ant-Density)

- 如果从t到t+1时刻，蚂蚁k经过路径ij，则

$$\Delta \tau_{ij}^k = Q$$

- 更新原则：第k只蚂蚁走完一个路径就更新

启发式算法—蚁群算法

如何更新信息素

 根据不同的规则我们可以将蚁群算法分为三种模型：

蚁量模型(Ant-Quantity)

- 如果从t到t+1时刻，蚂蚁k经过路径ij，则

$$\Delta \tau_{ij}^k = \frac{Q}{d_{ij}}$$

d_{ij} 是路径ij的长度

- 更新原则：第k只蚂蚁走完一个路径就更新

启发式算法—蚁群算法

如何更新信息素

 经过测试，蚁周模型(Ant-Cycle)效果更好，特别是在TSP问题中。

$$\Delta \tau_{ij}^k = \frac{Q}{L_k}$$

蚁群算法信息素更新示例

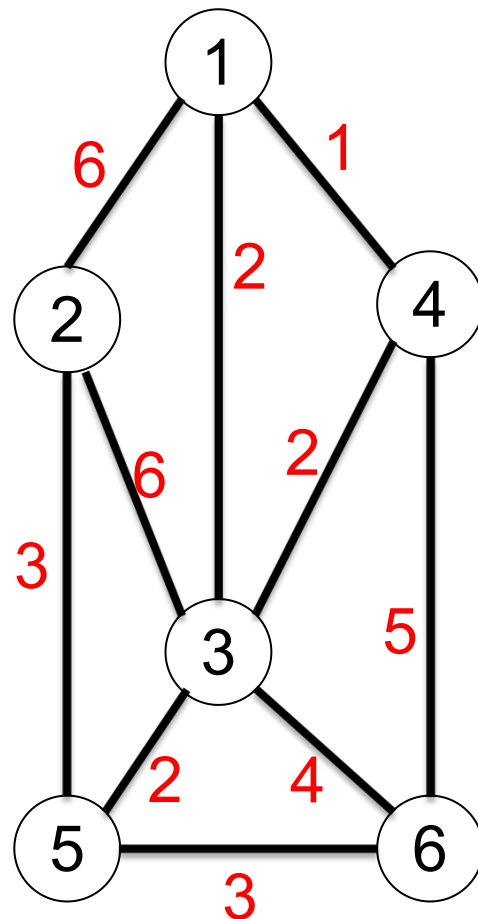
t次循环之后的信息素矩阵

	1	2	3	4	5	6
1	0	10	10	10	10	10
2	10	0	10	10	10	10
3	10	10	0	10	10	10
4	10	10	10	0	10	10
5	10	10	10	10	0	10
6	10	10	10	10	10	0

假设蚂蚁k走过的路径为1->2->5->6->3->4->1

则总路径长度 $L_k = 6 + 3 + 3 + 4 + 2 + 1 = 19$

假设信息素常量Q为38,则 $\Delta\tau_{ij}^k = 38/19 = 2$



蚁群算法信息素更新示例

t次循环之后的信息素矩阵

	1	2	3	4	5	6
1	0	10	10	10	10	10
2	10	0	10	10	10	10
3	10	10	0	10	10	10
4	10	10	10	0	10	10
5	10	10	10	10	0	10
6	10	10	10	10	10	0

信息素挥发
挥发因子
 $\rho = 0.7$

	1	2	3	4	5	6
1	0	3	3	3	3	3
2	3	0	3	3	3	3
3	3	3	0	3	3	3
4	3	3	3	0	3	3
5	3	3	3	3	0	3
6	3	3	3	3	3	0

添加第
k只蚂
蚁的在
路径上
释放的
信息素

	1	2	3	4	5	6
1	0	5	3	5	3	3
2	5	0	3	3	5	3
3	3	3	0	5	3	5
4	5	3	5	0	3	3
5	3	5	3	3	0	5
6	3	3	5	3	5	0

假设蚂蚁k走过的路径为1->2->5->6->3->4->1

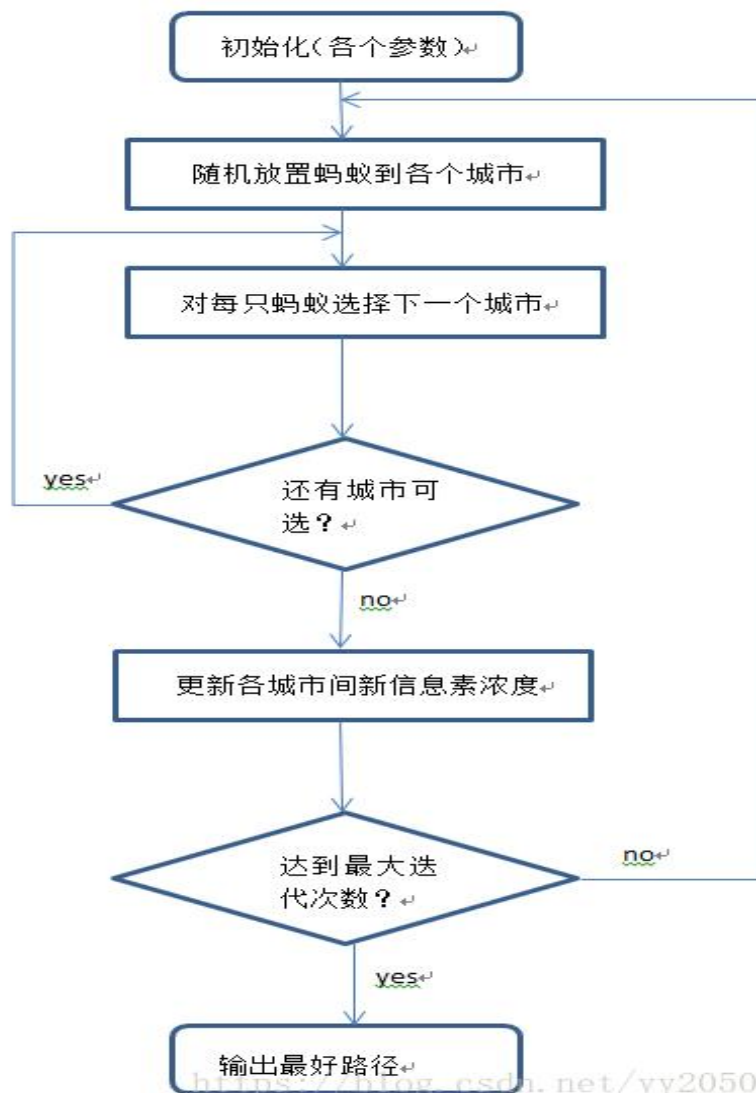
则总路径长度 $L_k=19$

信息素常量Q为38,则 $\Delta\tau_{ij}^k=2$

启发式算法—蚁群算法

算法流程

1. **初始化(各个参数):**
对各个参数进行初始化
2. **构建解空间:**
蚂蚁路径选择
3. **更新信息素:**
路径选择后, 进行信息素更新
4. **判断是否终止:**
判断是否结束寻找



蚁群算法流程

1. 初始化(各个参数):

在计算之初需要对相关的参数进行初始化，如蚂蚁数量 m 、信息素因子 α 、启发函数因子 β 、信息素挥发因子 ρ 、信息素常数 Q 、最大迭代次数 t 等等。

□ 蚂蚁数量 m :

- 过大——每条路径上信息素趋于平均，正反馈作用减弱，从而导致收敛速度减慢
- 过小——可能导致一些从未搜索过的路径信息素浓度减小为0，导致过早收敛，解的全局最优性降低

蚂蚁数量一般设置为目标数的1.5倍较为稳妥。

蚁群算法流程

1. 初始化(各个参数):

在计算之初需要对相关的参数进行初始化，如蚂蚁数量 m 、信息素因子 α 、启发函数因子 β 、信息素挥发因子 ρ 、信息素常数 Q 、最大迭代次数 t 等等。

□ 信息素常量 Q :

- 过大——会使蚁群的搜索范围减小容易过早的收敛，使种群陷入局部最优。
- 过小——每条路径上信息含量差别较小，容易陷入混沌状态

信息素常量根据经验一般取值在 $[10, 1000]$ 。

蚁群算法流程

1. 初始化(各个参数):

在计算之初需要对相关的参数进行初始化，如蚂蚁数量 m 、信息素因子 α 、启发函数因子 β 、信息素挥发因子 ρ 、信息素常数 Q 、最大迭代次数 t 等等。

□ 迭代次数 t :

- 过大——运算时间过长
- 过小——可选路径较少，使种群陷入局部最优。

最大迭代次数一般取 $[100, 500]$ ，例如常取200.

蚁群算法流程

1. 初始化(各个参数):

在计算之初需要对相关的参数进行初始化，如蚂蚁数量 m 、信息素因子 α 、启发函数因子 β 、信息素挥发因子 ρ 、信息素常数 Q 、最大迭代次数 t 等等。

□ 信息素因子 α :

- 过大——蚂蚁选择以前已经走过的路可能性较大，容易使随机搜索性减弱。
- 过小——蚁群易陷入纯粹的随机搜索，使种群陷入局部最优。

信息素因子 α 取值范围通常在 $[1, 4]$ 之间。

蚁群算法流程

1. 初始化(各个参数):

在计算之初需要对相关的参数进行初始化，如蚂蚁数量 m 、信息素因子 α 、启发函数因子 β 、信息素挥发因子 ρ 、信息素常数 Q 、最大迭代次数 t 等等。

□ 启发函数因子 β :

- 过大——虽然收敛速度加快，但是易陷入局部最优。
- 过小——蚁群易陷入纯粹的随机搜索，很难找到最优解。

启发函数因子 β 取值范围通常在 $[0, 5]$ 之间。

蚁群算法流程

1. 初始化(各个参数):

在计算之初需要对相关的参数进行初始化，如蚂蚁数量 m 、信息素因子 α 、启发函数因子 β 、信息素挥发因子 ρ 、信息素常数 Q 、最大迭代次数 t 等等。

□ 信息素挥发因子 ρ :

- 过大——信息素挥发较快，容易导致较优路径被排除。
- 过小——各路径上信息素含量差别较小，收敛速度降低。。

信息素挥发因子 ρ 取值范围通常在 $[0.2, 0.5]$ 之间

。

蚁群算法流程

2. 构建解空间:

将各个蚂蚁随机地放置于不同的出发点，对每个蚂蚁 k ($k=1, 2, \dots, m$)，计算其下一个待访问的城市，直到所有蚂蚁访问完所有的城市。

$$p_{ij}^k(t) = \frac{\left(\tau_{ij}(t)\right)^\alpha \left(\eta_{ij}(t)\right)^\beta}{\sum_{j \in N_i^k} \left(\tau_{ij}(t)\right)^\alpha \left(\eta_{ij}(t)\right)^\beta}$$

蚁群算法流程

3. 更新信息素：

计算各个蚂蚁经过的路径长度 L ，记录当前迭代次数中的最优解（最短路径）。同时，对各个城市连接路径上的信息素浓度进行更新。

$$\tau_{ij}(t+n) = (1-\rho)\tau_{ij}(t) + \sum_{k=1}^m \Delta\tau_{ij}^k$$

$$\Delta\tau_{ij}^k = \begin{cases} \frac{Q}{L_k} & \text{经过路径}ij \\ 0 & \text{不经过路径}ij \end{cases}$$

蚁群算法流程

4. 判断是否终止：

若迭代次数小于**最大迭代次数**则迭代次数加一，清空蚂蚁经过路径的记录表，并返回步骤二；否则终止计算，输出最优解。

蚁群算法中的终止条件：**是否达到迭代次数。**

一次迭代就是指 m 只蚂蚁都走完所有的城市，即存在 m 个搜索路径。

将所有路径进行比较，选择长度最短的路径，做出这一次迭代的可视化结果，同时迭代次数加1。

然后判断当前迭代次数是否等于设置的迭代次数。如果等于则停止迭代，否则进行下一次迭代。

启发式算法—蚁群算法

算法特点

自组织

- ✎ 算法执行由内部多个蚂蚁自行组织

并行计算

- ✎ 由多个蚂蚁同时进行路径寻找，属于分布式计算

鲁棒性好

- ✎ 可以修改适用于求解其他问题

蚁群算法求解其他组合优化问题

0-1背包问题

 目标：背包内装的物品价值最大

$$\max \sum_{i=0}^{N-1} v_i * x_i$$
$$x_i = \begin{cases} 1 & i \text{ 物品被装入} \\ 0 & i \text{ 物品未装入} \end{cases}$$

蚁群算法求解其他组合优化问题



0-1背包问题



约束：

- 1. 背包选择的物品总重量不能超过背包容量
- 2. 物品不可拆分（只能选择装或者不装）

$$\sum_{i=0}^{N-1} w_i * x_i \leq C$$

$$x_i = \begin{cases} 1 & i \text{ 物品被装入} \\ 0 & i \text{ 物品未装入} \end{cases}$$

蚁群算法求解其他组合优化问题



0-1背包问题



数学模型：

$$\begin{aligned} & \max \sum_{i=0}^{N-1} v_i * x_i \\ & s.t. \begin{cases} \sum_{i=0}^{N-1} w_i * x_i \leq C \\ x_i = \begin{cases} 1 & i \text{ 物品被装入} \\ 0 & i \text{ 物品未装入} \end{cases} \\ i = \{0, 1, 2, \dots, N-1\} \end{cases} \end{aligned}$$

蚁群算法求解其他组合优化问题



蚁群算法求解0-1背包问题



基本思想

- ❏ 将N个物品理解为蚁群觅食的路径上的点（TSP问题中的城市）。
- ❏ 蚂蚁每选择一个物品，则在该物品上留下信息素。
- ❏ 在物品选择时，利用蚁群路径选择公式进行计算，将其中的路径长度换成物品的价值。
- ❏ 蚂蚁每次遍历时，不再考虑回到原点，而是随机的不重复访问每个物品，在总重量小于背包的约束下得到一组解。

蚁群算法求解其他组合优化问题



蚁群算法求解0-1背包问题



基本思想

- ❏ 将N个物品理解为蚁群觅食的路径上的点（TSP问题中的城市）。
- ❏ 蚂蚁每选择一个物品，则在该物品上留下信息素。
- ❏ 在物品选择时，利用蚁群路径选择公式进行计算，将其中的路径长度换成物品的价值。
- ❏ 蚂蚁每次遍历时，不再考虑回到原点，而是随机的不重复访问每个物品，在总重量小于背包的约束下得到一组解。

蚁群算法求解0-1背包问题主要步骤

1. 初始化(各个参数):

对相关的参数进行初始化，如蚂蚁数量 m 、信息素因子 α 、启发函数因子 β 、信息素挥发因子 ρ 、信息素常数 Q 、最大迭代次数 t 等等。

对于0-1背包问题，在这一步还需要对以下参数（变量进行初始化）：

- ① 物品数量 N
- ② 物品价格 w_i , 和物品价值 v_i . $i=\{0,1,2\ldots,N-1\}$
- ③ 背包容量 C
- ④ 蚁群路径记录函数 x_i . $i=\{0,1,2\ldots,N-1\}$

实际编程时，还需要其他很多变量，这里不一一列举

蚁群算法求解0-1背包问题主要步骤

2. 构建解空间：

将各个蚂蚁随机地放置于不同的出发点，对每个蚂蚁 k ($k=1, 2, \dots, m$)，计算其下一个待访问的城市，直到**所有蚂蚁访问完所有的城市**。

对于0-1背包问题，在这一步需要作如下改变：

- ① 由于不需要寻找一个回到起点的路径，因此，蚂蚁的初始点是相同的，都是一个虚拟的，无意义的出发点
- ② 下一个选择的城市这里就变成了下一个选择的物品
- ③ 蚂蚁根据背包容量的约束选择物品，当所有物品都遍历完成后，实际装入背包的就是它此次迭代的可行解
(注意：有可能一个物品都没有装入)

蚁群算法求解0-1背包问题主要步骤

2. 构建解空间:

对于0-1背包问题，在这一步需要作如下改变：

- ① 由于不需要寻找一个回到起点的路径，因此，蚂蚁的初始点是相同的，都是一个虚拟的，无意义的出发点
- ② 下一个选择的的城市这里就变成了下一个选择的物品
- ③ 蚂蚁根据背包容量的约束选择物品，当所有物品都遍历完成后，实际装入背包的就是它此次迭代的可行解
(注意：有可能一个物品都没有装入)

$$p_{ij}^k(t) = \frac{\left(\tau_{ij}(t)\right)^{\alpha} \left(\eta_{ij}(t)\right)^{\beta}}{\sum_{j \in N_i^k} \left(\tau_{ij}(t)\right)^{\alpha} \left(\eta_{ij}(t)\right)^{\beta}}$$

物品的价值

蚁群算法求解0-1背包问题主要步骤

3. 更新信息素：

计算各个蚂蚁经过的路径长度 L ，记录当前迭代次数中的最优解（最短路径）。同时，对各个城市连接路径上的信息素浓度进行更新。

$$\tau_{ij}(t+n) = (1-\rho)\tau_{ij}(t) + \sum_{k=1}^m \Delta\tau_{ij}^k$$
$$\Delta\tau_{ij}^k = \begin{cases} \frac{Q}{L_k} & \text{经过路径} ij \\ 0 & \text{不经过路径} ij \end{cases}$$

对于0-1背包问题，在这一步需要作如下改变：

- ① 所有蚂蚁都完成本次迭代后，进行更新，采用的是蚁周模型，记录总价值最大的蚂蚁编号
- ② 信息素浓度计算公式需要改进为 $\Delta\tau_{ij}^k = QV_k$ ——

蚂蚁 k 装入
物品总价值

蚁群算法求解0-1背包问题主要步骤

4. 判断是否终止：

若迭代次数小于**最大迭代次数**则迭代次数加一，清空蚂蚁经过路径的记录表，并返回步骤二；否则终止计算，输出最优解。

对于0-1背包问题，在这一步需要作如下改变：

- ① 迭代结束后，选择最后一次迭代背包装入总价值最大的那只蚂蚁的路径作为最优解

启发式算法—蚁群算法

缺点和不足

1.收敛速度慢

-  蚁群算法中信息素初值相同，选择下一个节点时倾向于随机选择。虽然随机选择能探索更大的任务空间，有助于找到潜在的全局最优解，但是需要较长时间才能发挥正反馈的作用，导致算法初期收敛速度较慢。

启发式算法—蚁群算法

缺点和不足

2. 局部最优问题。

-  蚁群算法具有正反馈的特点，初始时刻环境中的信息素完全相同，蚂蚁几乎按随机方式完成解的构建，这些解必然会存在优劣之分。在信息素更新时，蚁群算法在较优解经过的路径上留下更多的信息激素，而更多的信息激素又吸引了更多的蚂蚁，这个正反馈的过程迅速地扩大初始的差异，引导整个系统向最优解的方向进化。虽然正反馈使算法具有较好的收敛速度，但是如果算法开始得到的较优解为次优解，那么正反馈会使次优解很快占据优势，使算法陷入局部最优，且难以跳出局部最优。

启发式算法—蚁群算法

缺点和不足

3. 优化能力问题。

-  蚁群算法中参数众多并且具有一定的关联性，虽然蚁群算法在很多领域都有广泛应用，但是参数选择更多是依赖经验和试错，不恰当的初始参数会减弱算法的寻优能力。当进行路径规划时，为避免形成环形路径或者重复访问某些节点在算法中设置禁忌表，但是禁忌表很容易造成“死锁”现象，减少种群中的有效蚂蚁数量，降低算法的优化效率。

启发式算法—蚁群算法

缺点和不足

4. 种群多样性与收敛速度的矛盾

-  种群多样性对应于候选解在问题空间的分布。个体分布越均匀，种群多样性就越好，得到全局最优解的概率就越大，但是寻优时间就越长；个体分布越集中，种群多样性就越差，不利于发挥算法的探索能力。正反馈加快了蚁群算法的收敛速度，却使算法较早地集中于部分候选解，因此正反馈降低了种群的多样性，也不利于提高算法的全局寻优能力。。

启发式算法—蚁群算法

动手试一试

 用任意编程语言实现蚁群算法解决0-1背包问题。